Retries and Exponential Backoff in Practice

retry once, at one layer, slowly

Watch the 4-min video

In one line: retry only transient failures of idempotent operations, with capped exponential backoff and jitter, at a single layer, under a retry budget, so retries rescue brief blips instead of multiplying load on a dependency that is already struggling.

1 How a blip becomes an outage

The inventory database slows down for two seconds. Clients time out and retry. The gateway retries too, and so does the service. The database now receives many times its normal load at the moment it is weakest, can't recover, and a two-second blip becomes an hour-long **retry storm**.

2 What to retry

Failure	Retry?	Why
connection refused / reset, DNS blip	yes	transient; the request probably never ran
timeout	only if idempotent	the request may have succeeded on the server
503 Service Unavailable, 429 Too Many Requests	yes, later	overload; honour the `Retry-After` header
400, 401, 403, 404, 422	no	permanent: the same request fails the same way
non-idempotent POST (charge a card)	only with an idempotency key	otherwise a retry can charge twice

Idempotent means doing it twice has the same effect as once: GET, PUT, DELETE by design; a payment only if the server deduplicates on an **idempotency key** sent by the client.

3 Backoff, jitter and limits

$$sleep = random(0,\ min(cap,\ base \times 2^{attempt}))$$

Never retry immediately. **Exponential backoff** waits 100, 200, 400 ms... up to a cap. **Jitter** randomises each wait, so thousands of clients that failed at the same moment don't all retry at the same moment. The formula above is AWS's "full jitter". Limit attempts to two or three, and stop when the request's deadline (lesson 4) runs out.

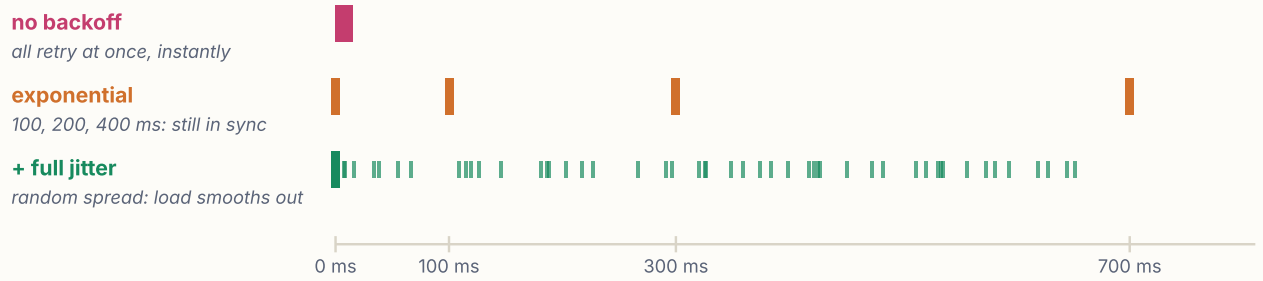


Figure 1. Backoff spaces retries out; jitter de-synchronises clients so retries arrive as a smooth trickle instead of waves.

4 Retry at one layer, under a budget

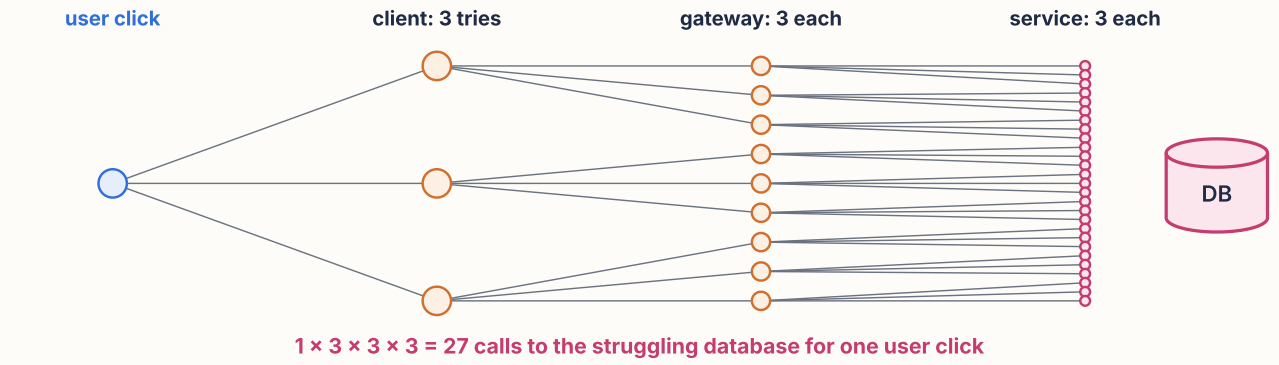


Figure 2. Each layer's retries multiply the layers below it. Three layers × three attempts = 27 calls.

Retry at **one layer**, usually the one closest to the failing call, and let the others pass errors through. Then add a **retry budget**: retries may be at most a small fraction of normal traffic, e.g. **10%**. When failures are rare, every failure gets retried; when most requests fail, the budget runs out and retries stop, so they can't double or triple the load. gRPC's retry throttling and Envoy's retry budgets implement this idea.

Combine with a **circuit breaker** (lesson 6): once it opens, calls fail fast with no retries at all. Servers can help with `Retry-After` or an explicit "overloaded, don't retry" signal.

WORKED EXAMPLE: THE 27× STORM

The inventory DB slows down. Client, gateway and service each make up to 3 attempts, so every click becomes up to **27 queries**. Fix: the client and gateway pass errors straight back; only the service retries, with backoff 100 → 200 ms plus jitter, at most 3 attempts inside a 1.5 s deadline, and a retry budget of 10%. During the incident the extra load on the database is now **at most 10%** instead of up to 27×, and it recovers.

COMMON MISTAKES

- Retrying 4xx errors, which will never succeed.
- Retrying payments or other side effects without an idempotency key.
- Backoff without jitter, so clients retry in synchronised waves.
- Retries configured independently in the SDK, the gateway and the service mesh (lesson 27).

5 Key terms

Transient failure: temporary error likely to succeed on retry

Idempotent: doing it twice = doing it once

Exponential backoff: wait doubles after each failed attempt, up to a cap

Jitter: random part of the wait to de-sync clients

Retry amplification: attempts multiplying across layers

Retry budget: cap on retries as a share of normal traffic

6 Interview questions

Q1. Which errors should you retry? BASIC

Transient ones (connection resets, timeouts, 503, 429) on idempotent operations; never permanent errors like 400 or 404.

Q2. Why add jitter to backoff? BASIC

Clients that failed together would otherwise retry together in synchronised waves; randomising the wait spreads the load.

Q3. How do you avoid retry storms? SCENARIO

Retry only transient errors on idempotent operations, use capped exponential backoff with jitter, retry at a single layer, enforce a retry budget (e.g. ≤ 10%) and add a circuit breaker.

Q4. A payment call timed out. Is it safe to retry? SCENARIO

Only if the call is idempotent: send an idempotency key so the provider returns the original result instead of charging again.

Q5. Why is retrying at every layer dangerous? TRICKY

Attempts multiply: three layers × three attempts = 27 calls to the failing dependency for one user request, exactly when it's weakest.

Q6. Why is a retry budget better than a fixed attempt count? TRICKY

A fixed count retries every failure even during a full outage, adding load; a budget allows retries when failures are rare and switches them off when they're widespread.

Next: SD-K-006 · Circuit Breaker backendroadmap.com